MANCHESTER 1824
The University of Manchester

word2vec, RNNs, and Transformers

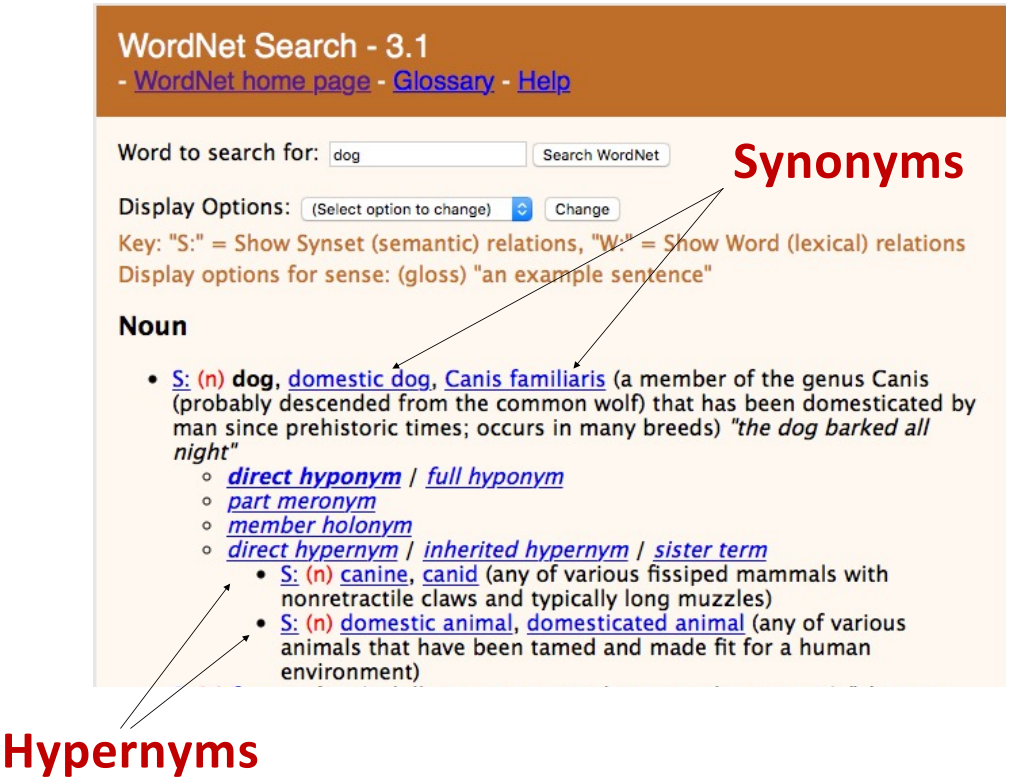
Chenghua Lin
Department of Computer Science
The University of Manchester

MANCHESTER 1824
The University of Manchester

Evolution of Language Models

MANCHESTER 1824
The University of Manchester

How can we represent word meaning?



- Traditional approach: symbolic lexical resources (e.g., WordNet)
- Represent meaning via explicit relations:
  - synonymy (dog ↔)
  - hypernymy (dog → animal)
- Meaning is **discrete**, **manually defined**, and **relational**

MANCHESTER 1824
The University of Manchester

Limitations of Symbolic Lexical Resources

- **Static and incomplete**
  - Cannot keep pace with new words, senses, or usage
- **Expensive to maintain**
  - Requires expert human annotation
- **Limited notion of similarity**
  - No natural notion of *graded* similarity
  - Cannot capture usage-based meaning differences

MANCHESTER 1824
The University of Manchester

Discrete Representation

- Words treated as atomic symbols (e.g., *one-hot encoding*).

motel [0, 0, 0, 0, 0, 1, 0, 0, 0, …, 0, 0]

hotel [0, 0, 1, 0, 0, 0, 0, 0, 0, …, 0, 0]

- **Vocabulary size = dimensionality** (can be very large)
- **Key limitation**: all words are equally dissimilar
  - cosine(motel, hotel) = 0

MANCHESTER 1824
The University of Manchester

Distributional Semantics

"You shall know a word by the company it keeps."

(J. R. Firth 1957)

*A **bank** is a financial institution that accepts deposits from the public and creates credit*

*In geography, the word **bank** generally refers to the land alongside a body of water.*

- Word meaning inferred from **contextual usage**
- Same word, different contexts → different meanings
- Meaning becomes statistical rather than symbolic

MANCHESTER 1824
The University of Manchester

Context-based Word Representations

- Build a word–context co-occurrence matrix
- Each word represented by its surrounding words
- **Rows** = target words; **Columns** = context words; **Values** = frequency counts

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

MANCHESTER 1824
The University of Manchester

Issues with raw co-occurrence count

- Extremely high-dimensional
- Sparse and memory-intensive
- Sensitive to corpus size and frequency effects

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

MANCHESTER 1824
The University of Manchester

Low Dimensional Dense Word Embeddings

- Represent each word as a **dense vector**
- Fixed dimensionality (typically **100–300 for word2vec**)
- Learned **automatically from data**

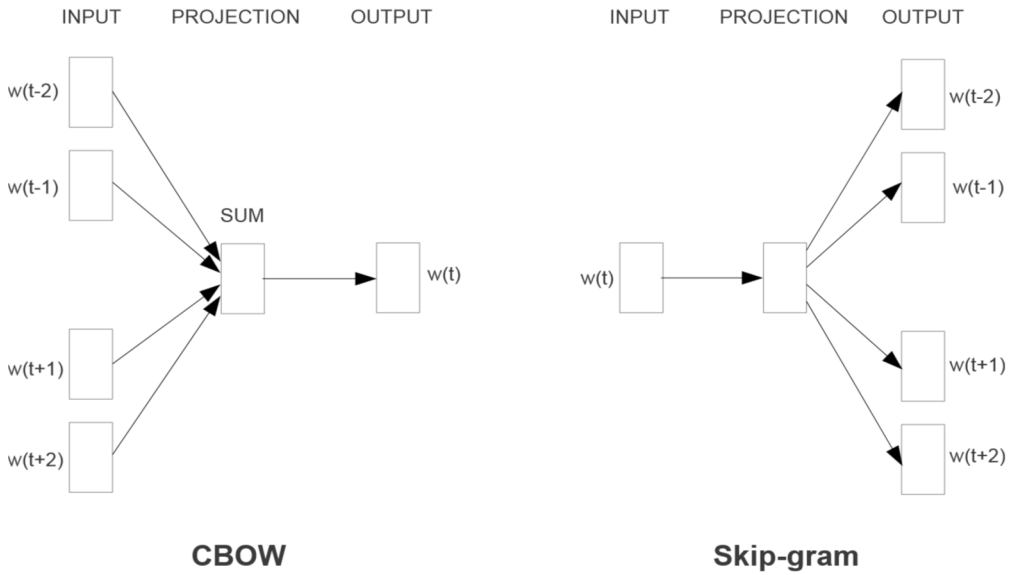
Core Idea of word2vec

- Learn word embeddings by **predicting context**
- Replace explicit co-occurrence counts with a **prediction task**

MANCHESTER 1824
The University of Manchester

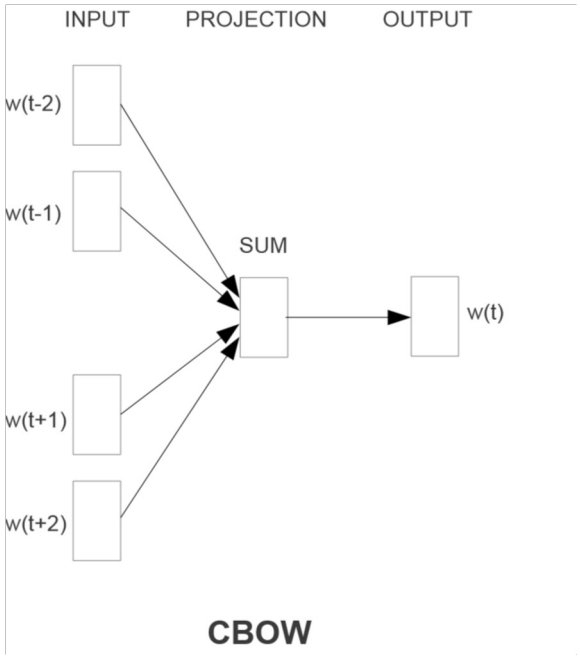
Word2vec Architectures

- Two training objectives
  - **CBOW**: predict target word from context
  - **Skip-gram**: predict context from target word



MANCHESTER 1824
The University of Manchester

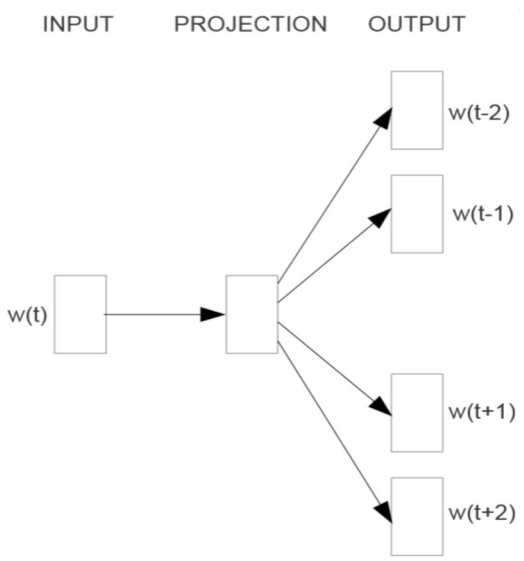
Continuous Bag-of-Words (CBOW)



- **Input**: surrounding context words
- **Output**: target word
- Context vectors are **summed or averaged**
- Efficient for frequent words

"Given the context, what word fits here?"

Skip-gram



Skip-gram

- **Input**: target word
- **Output**: surrounding context words
- Generates multiple training pairs per word
- Often performs better than CBOW for rare words

"Given this word, what words tend to appear nearby?"

MANCHESTER 1824
The University of Manchester

CBOW vs. Skip-gram

CBOW: predict *apple* from {eat, an, every, day}

Skip-gram: predict {eat, an, every, day} from *apple*

MANCHESTER 1824
The University of Manchester

Training objective (CBOW)

- Objective function

$$\prod_{w \in C} P(w \mid \text{context}(w))$$

- Taking logarithm, we have

$$\mathcal{L} = \sum_{w \in C} \log P(w \mid \text{context}(w))$$

MANCHESTER 1824
The University of Manchester

Applications of Word Embeddings

- Dependency parsing
- Named entity recognition
- Document classification
- Sentiment Analysis
- Paraphrase Detection
- Word Clustering
- Machine Translation

MANCHESTER 1824
The University of Manchester

Word Analogies

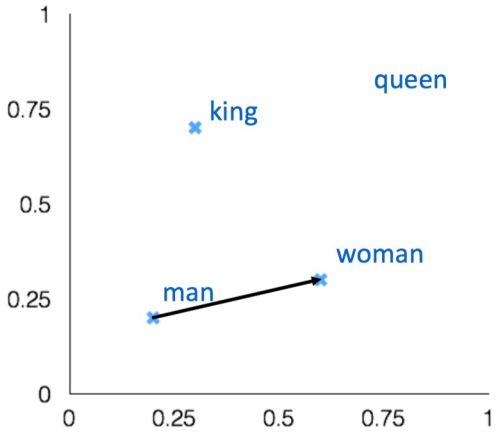
- Test for **linear regularities** in embedding space
- Relations captured as vector offsets
- Empirical property observed by Mikolov et al. (2014)

a:b :: c:?

$$d = \arg\max_x \frac{(w_b - w_a + w_c)^T w_x}{||w_b - w_a + w_c||}$$

man:woman :: king:?

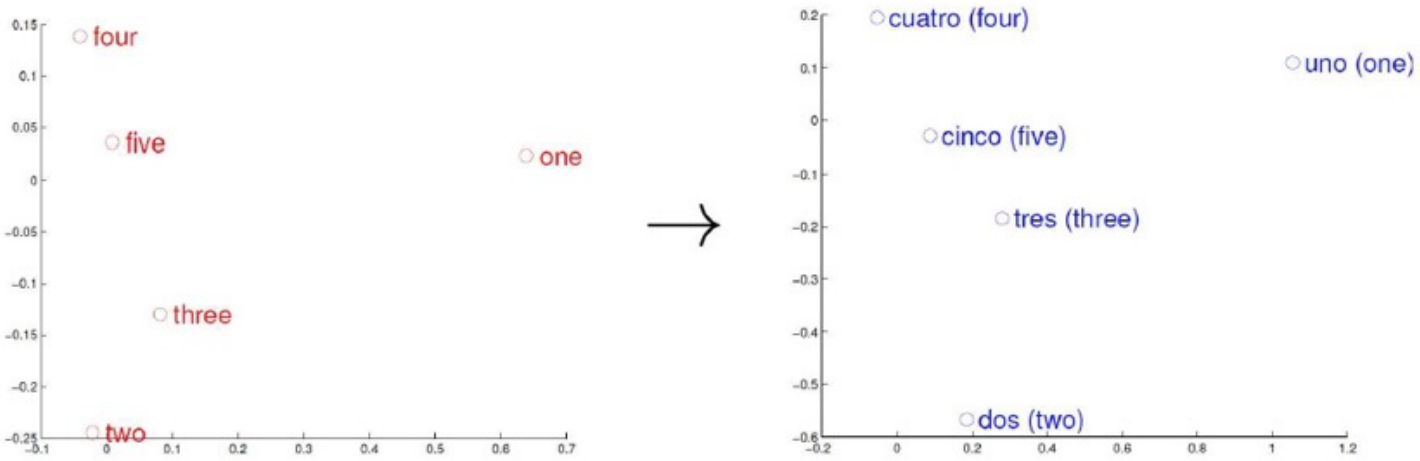
+	king	[0.30 0.70]
-	man	[0.20 0.20]
+	woman	[0.60 0.30]
	queen	[0.70 0.80]



MANCHESTER 1824
The University of Manchester

Machine Translation

- Similar concepts align geometrically across languages
- Enables bilingual lexicon induction
- Precursor to modern multilingual embeddings

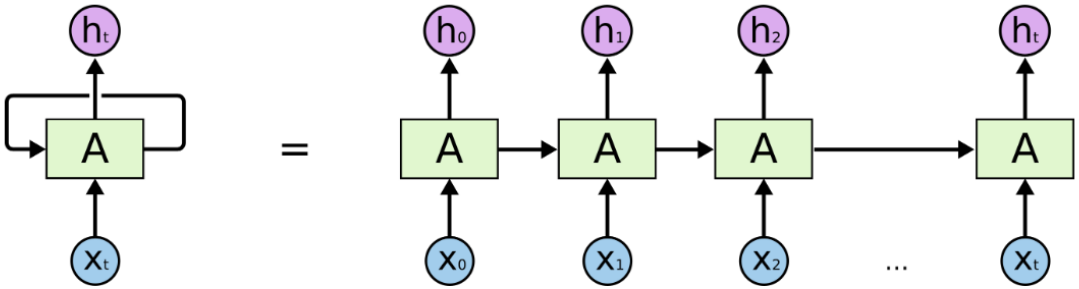


MANCHESTER 1824
The University of Manchester

Evolution of Language Models

MANCHESTER 1824
The University of Manchester

Recurrent Neural Networks (RNNs)



An unrolled recurrent neural network.

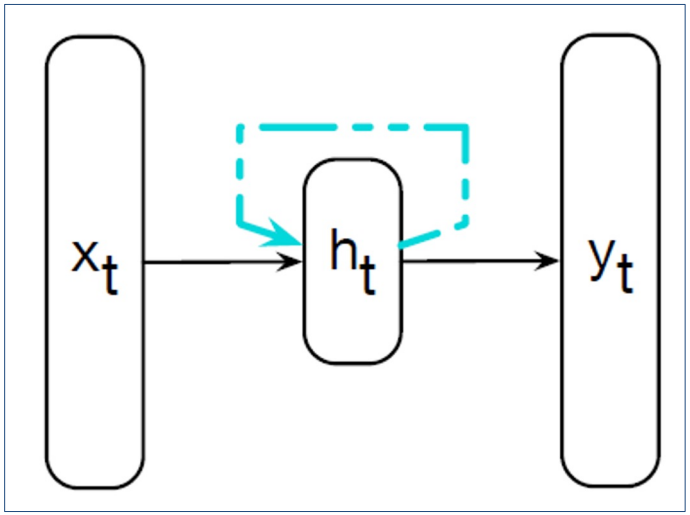
Source: http://clah.github.io/posts/2015-08-Understanding-LSTMs/

MANCHESTER 1824
The University of Manchester

Recurrent Neural Networks (RNNs) as LMs

Recurrent Neural Network

- A network that contains a cycle, i.e., the hidden state is fed back and used at the next time step
- The hidden layer has a recurrent connection: the hidden state at time t depends on
  – the current input x_t, and
  – the hidden state from the previous time step h_{t-1}



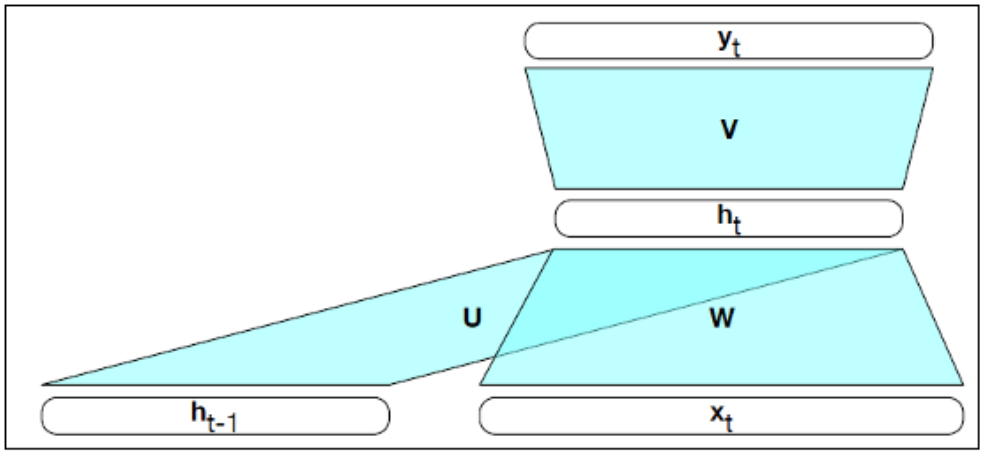
An Elman network (simple RNN)

MANCHESTER 1824
The University of Manchester

Recurrent Neural Networks (RNNs) as LMs

Simple RNN

- The sequence is processed one token at a time, from left to right
- At each time step t, the hidden state is updated using:
  – the current input x^t
  – the previous hidden state $h_{(t-1)}$
- The hidden state h_t serves as a summary of past inputs and is used to compute the output at time t.



A closer look at a simple RNN

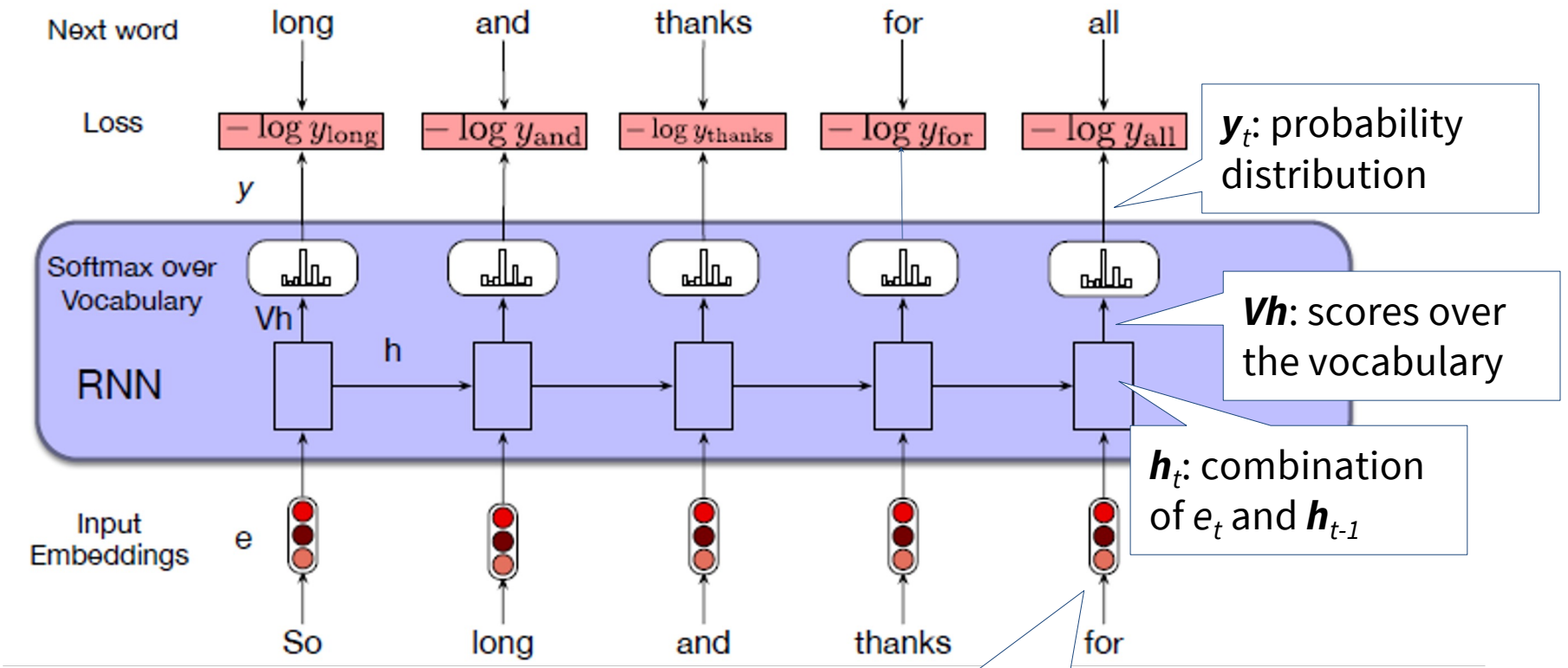
MANCHESTER 1824
The University of Manchester

Recurrent Neural Networks (RNNs) as LMs

- RNN language models
  - Process an input sequence one word at a time
  - To predict the next word from the current word and the previous hidden state
  - Model the next-word probability
- Computation is sequential, limiting parallelization

MANCHESTER 1824
The University of Manchester

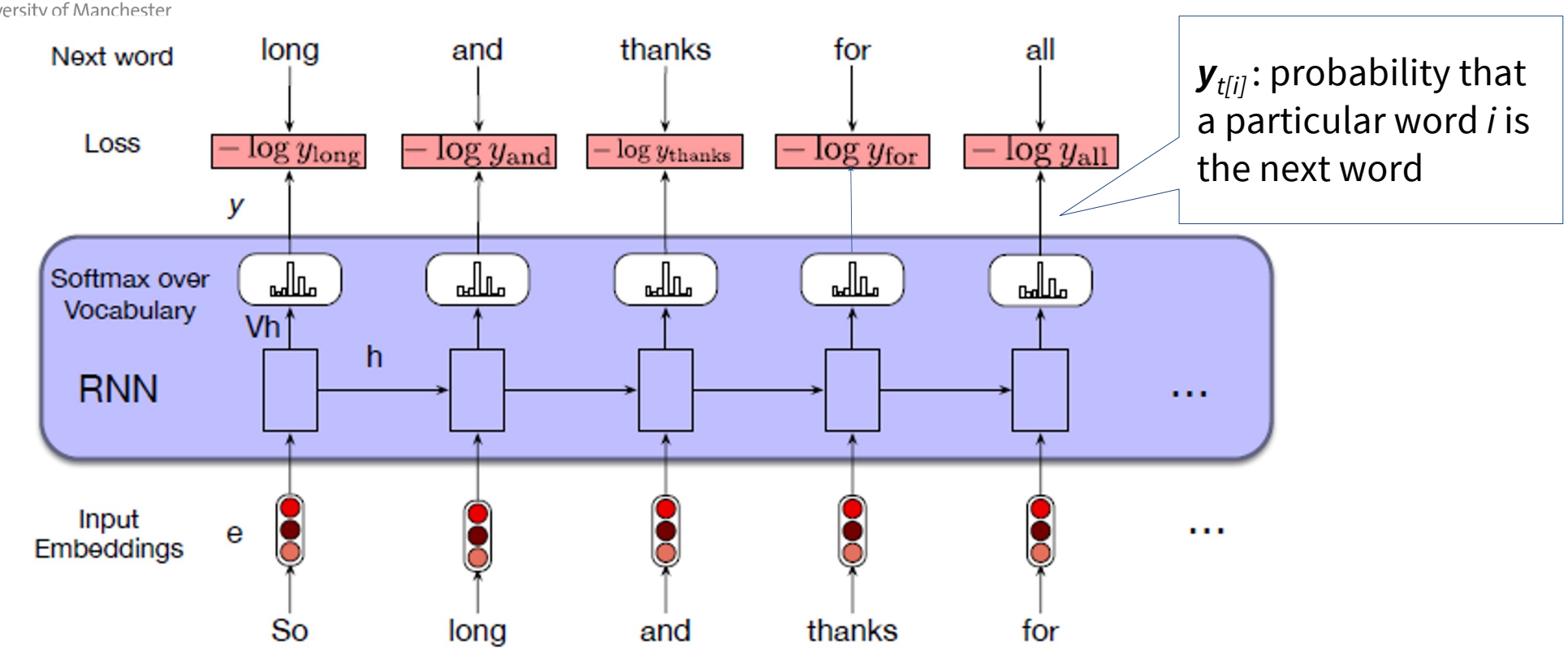
Recurrent Neural Networks (RNNs) as LMs



Let *t* be the current time step; w_t is the current word

MANCHESTER 1824
The University of Manchester

Recurrent Neural Networks (RNNs) as LMs



Let *t* be the current time step; w_t is the current word

MANCHESTER 1824
The University of Manchester

Recurrent Neural Networks (RNNs) as LMs

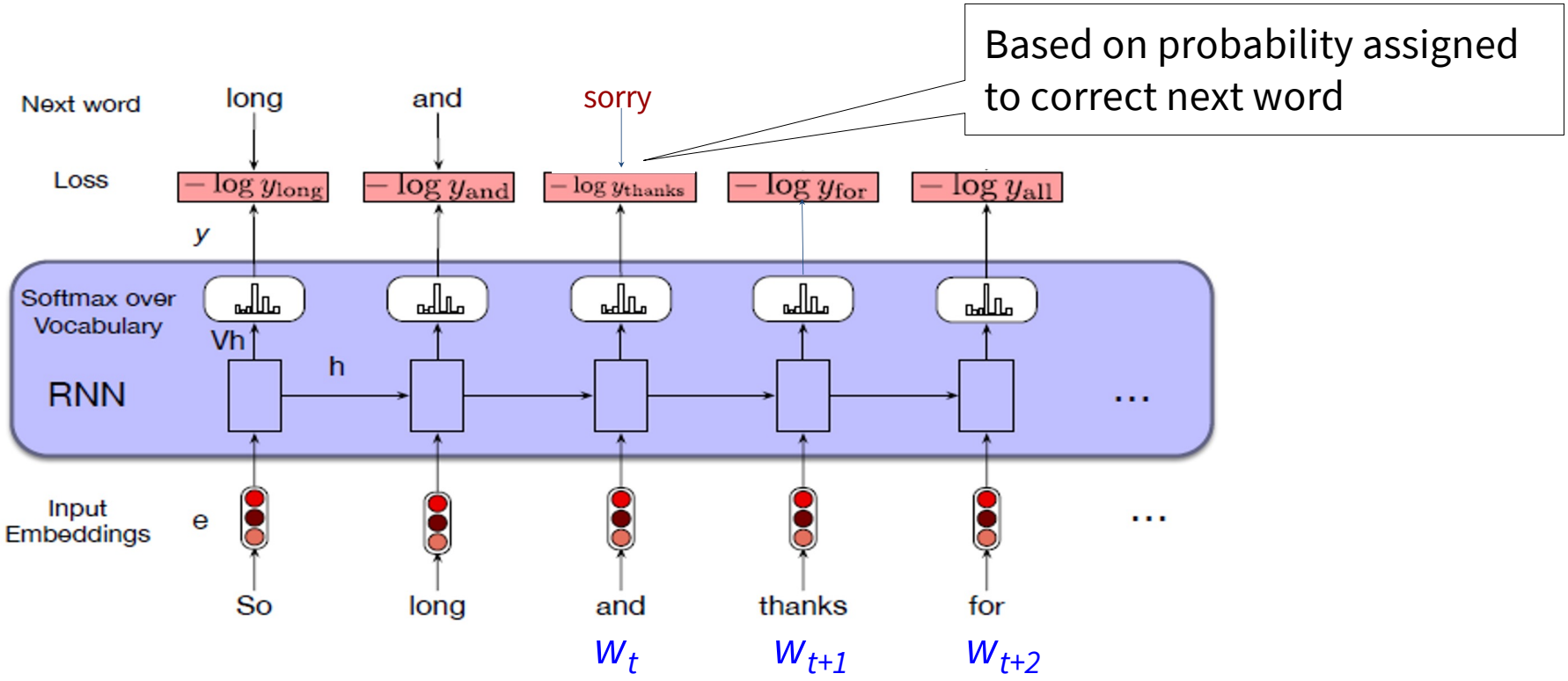
- Training an RNN as a language model
  - based on a corpus, model learns to predict the next word at each time step t
  - model minimises the error in predicting the correct next word
  - error/loss: the difference between a predicted probability distribution and the correct distribution, i.e., cross-entropy, where:

correct distribution: one-hot vector for the vocabulary, where the actual next word = 1 and the rest are 0

MANCHESTER 1824
The University of Manchester

Recurrent Neural Networks (RNNs) as LMs

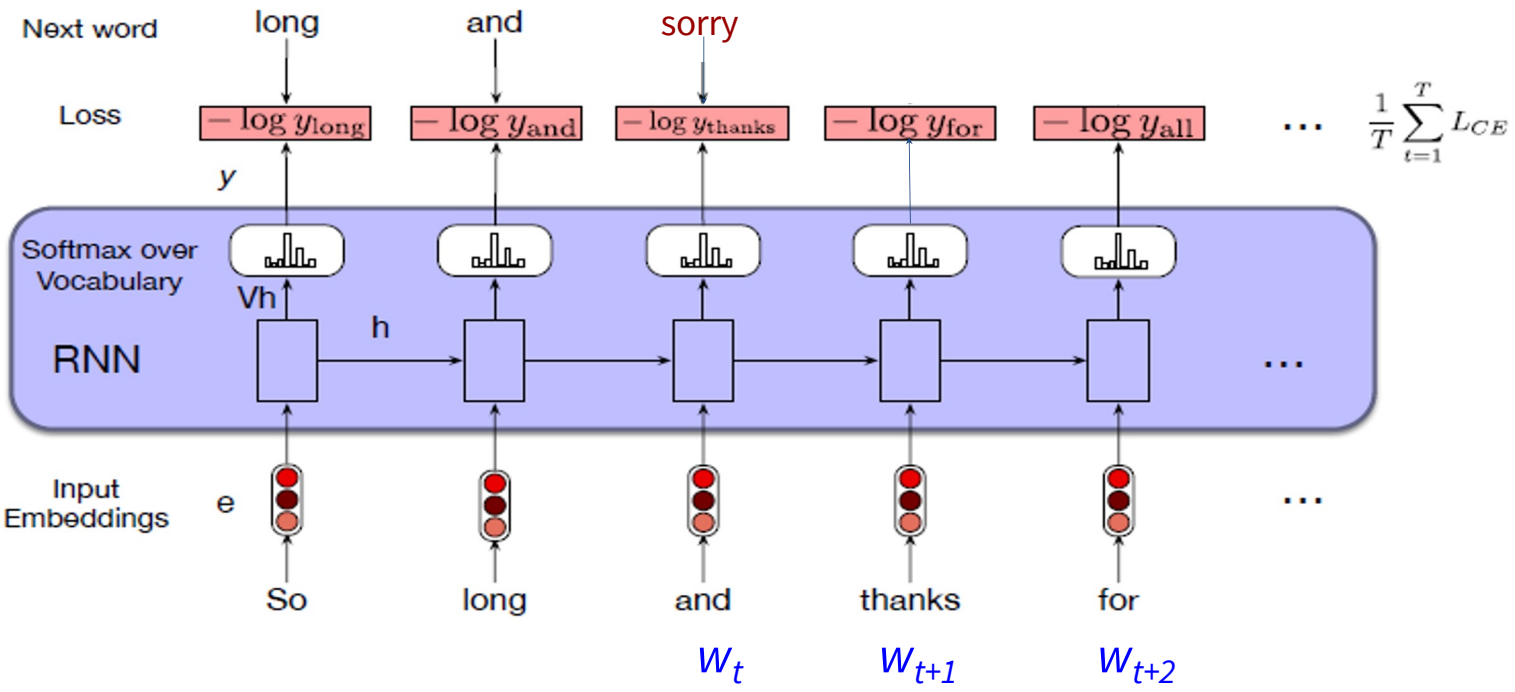
Teacher forcing: the model is given the correct history sequence (rather than what was predicted in the previous time step)



MANCHESTER 1824
The University of Manchester

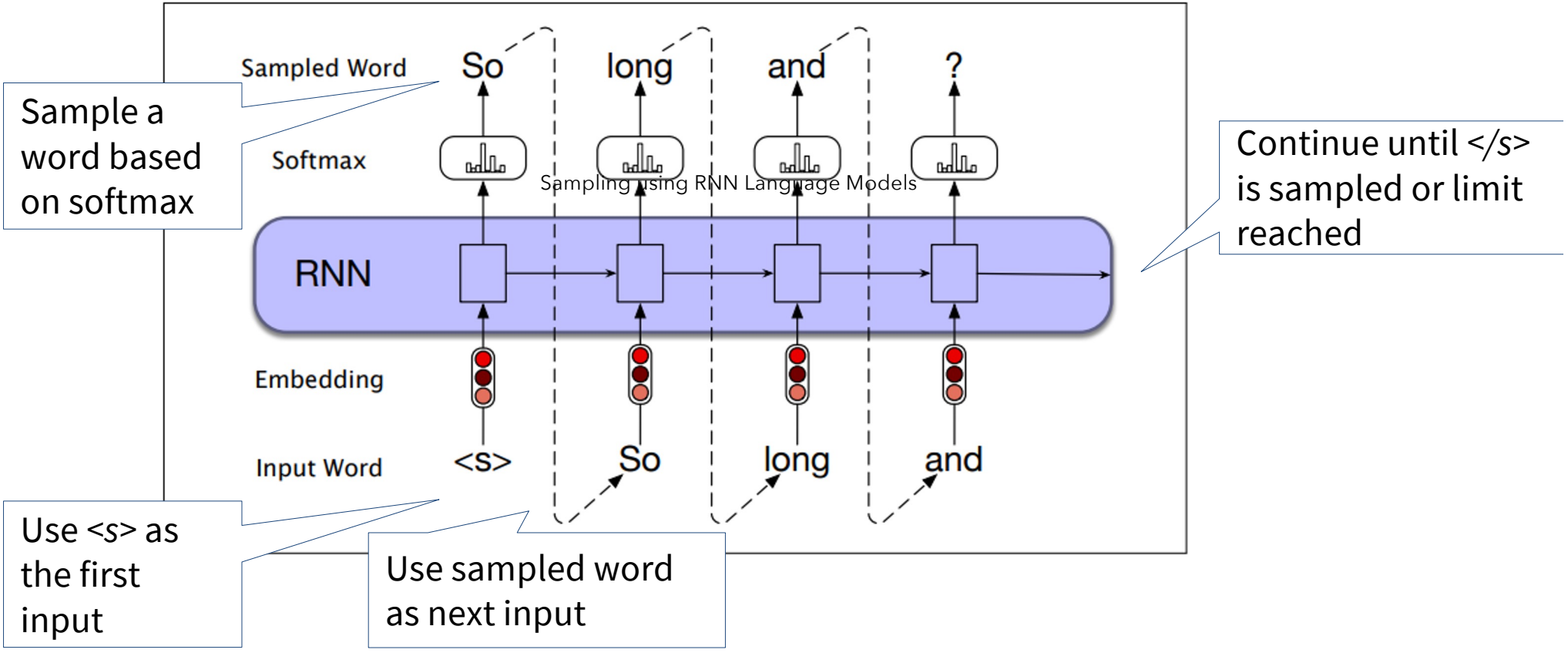
Recurrent Neural Networks (RNNs) as LMs

Gradient descent: used to adjust the weights in the RNN, in order to minimise the CE loss (L_{CE}) averaged over the sequence



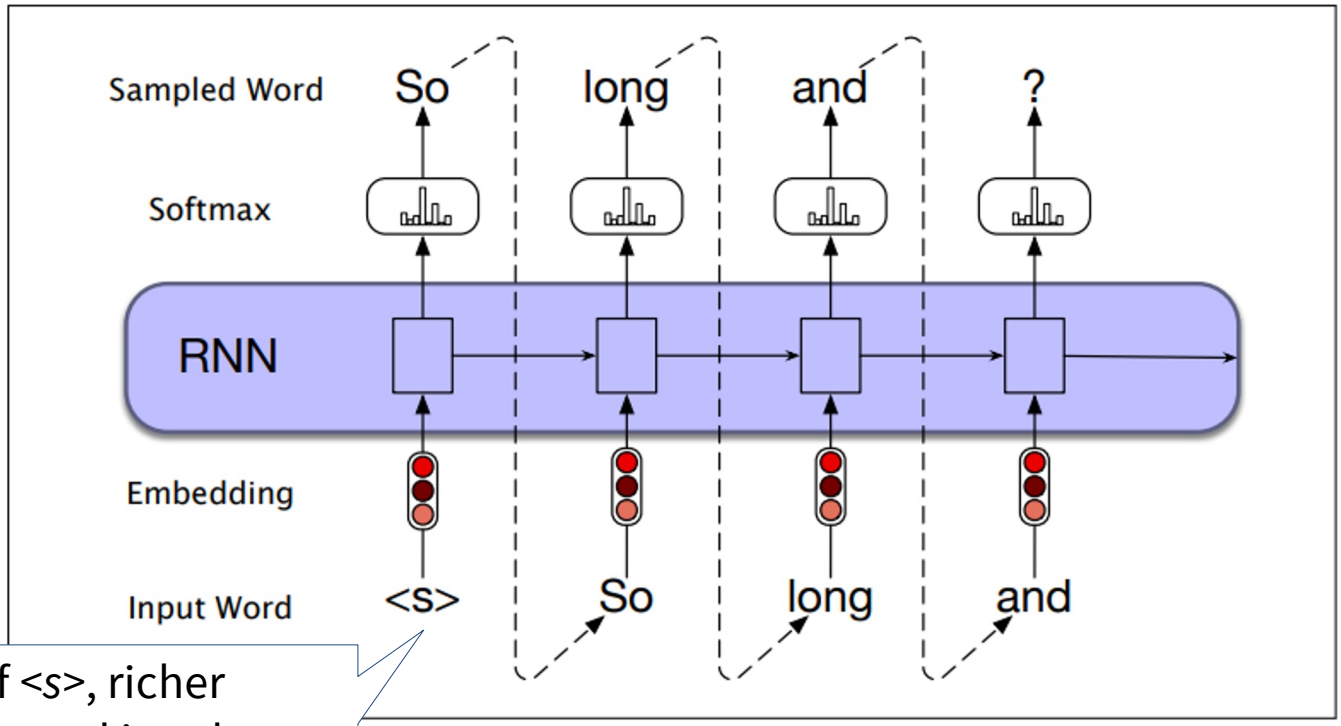
Sampling using RNN Language Models

Autoregressive generation: the word generated at each time step t is conditioned on the word selected by the RNN from the previous time step t-1



Sampling using RNN Language Models

Autoregressive generation: used in many other applications (apart from predicting the next word), e.g., machine translation, summarisation



Instead of <s>, richer context is used in other applications

MANCHESTER 1824
The University of Manchester

Quick Comparison

- Key difference between *n*-gram and RNN language models:
  - *n*-gram language models incorporate information from limited context only (i.e., preceding n-1 tokens)
  - RNN language models: the hidden state can incorporate information from all the preceding words

MANCHESTER 1824
The University of Manchester

Limitations of RNNs

- Process input sequentially, slow training and inference
- Struggle with long-range dependencies despite LSTM/GRU improvements.
- Limited parallelisation during training, so hard to scale on large datasets.
- Gradient vanishing/exploding issues over long sequences.
- Difficult to capture global context effectively

There is another type, i.e., transformer language models, up next!

MANCHESTER 1824
The University of Manchester

Turning Point – Transformer

- Proposed by Vaswani et al. (2017) in "*Attention is All You Need*".
- Eliminates recurrence, i.e., no step-by-step token processing.
- Enables parallelization: trains faster by processing entire sequences simultaneously.
- Leverages *self-attention* to model long-range dependencies.
- Achieves SOTA results in machine translation and beyond

MANCHESTER 1824
The University of Manchester

Evolution of Language Models

MANCHESTER 1824
The University of Manchester

Transformer

Encoder Only

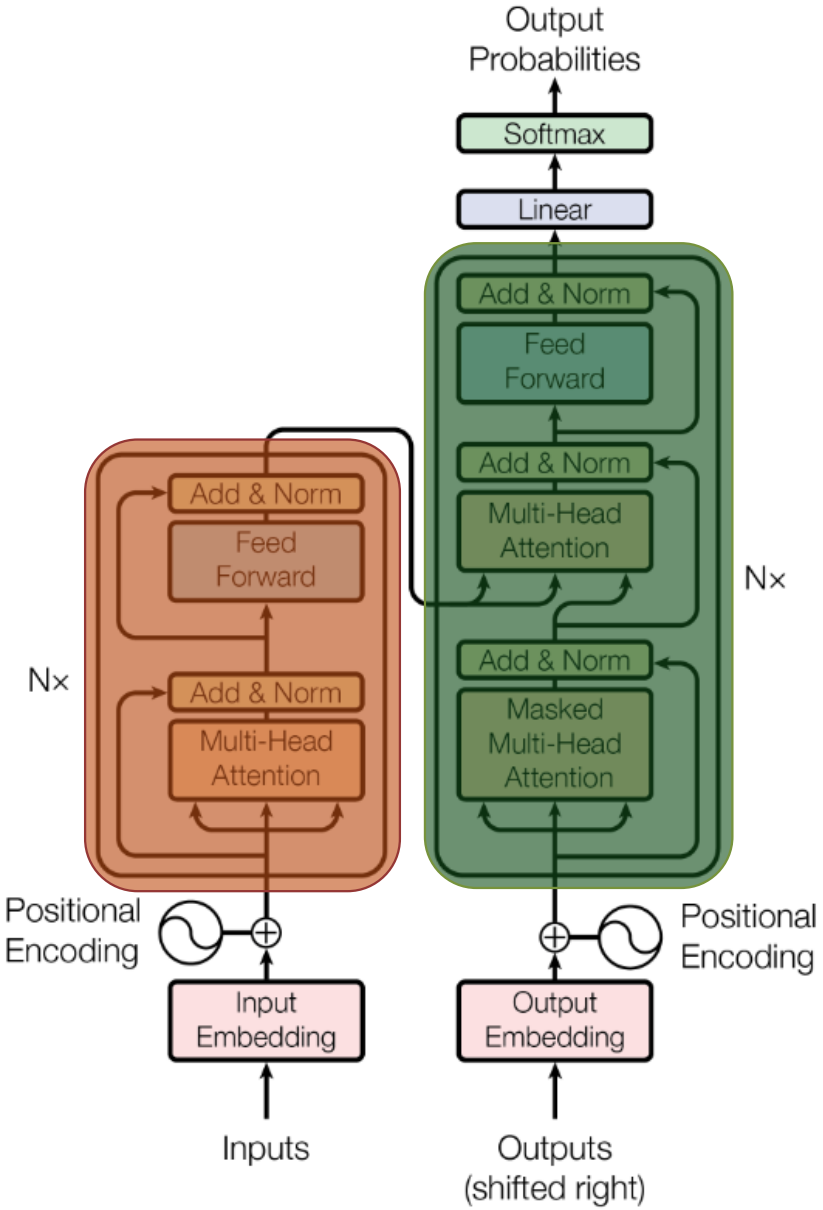
Uses: NLU, classification, feature extraction
Examples: BERT, RoBERTa, DeBERTa

Encoder-Decoder

Uses: NLG, translation, summarisation, seq2seq mapping
Examples: T5, Flan-T5, BART

Decoder Only

Uses: NLG, translation, summarisation, completion
Examples: GPT-x, LLaMa, every other new model today



Transformers

Transformer: a neural architecture built entirely on attention mechanisms.

Credits: The Illustrated Transformer – Jay Alammar

MANCHESTER 1824
The University of Manchester

Transformer Architecture

Core Components

- **Encoder Stack**: produces contextual representations of the input.
- **Decoder Stack**: uses the encoder's output and previous decoder outputs to generate the target sequence.
- **Self-Attention Mechanism**: allows the model to focus on different parts of the input sequence when processing each token.
- **Feed-Forward Neural Networks**: apply nonlinear transformation per token.

***Key idea**: no recurrence, no convolution — only attention.*

MANCHESTER 1824
The University of Manchester

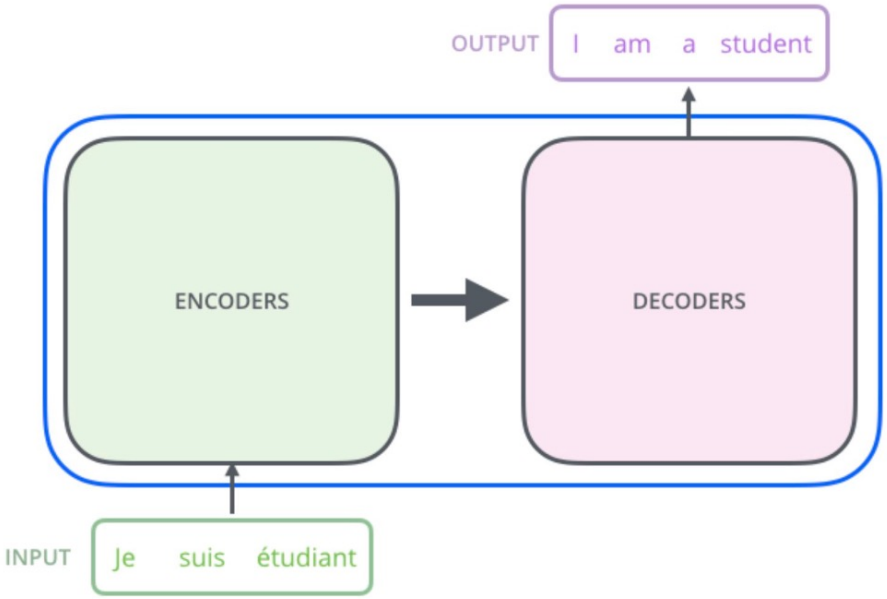
Why Self-Attention Replaced RNNs

- In RNNs:
  - Information between distant tokens flows through $O(n)$ steps
  - Computation is sequential, i.e. hard to parallelize
- In Transformers:
  - Self-attention allows each token to directly attend to all other tokens in the sequence, producing contextualised representations
  - Computation within each layer is parallelizable across sequence positions
- Trade-off:
  - Attention time and memory scale **quadratically** with sequence length $O(n^2)$

MANCHESTER 1824
The University of Manchester

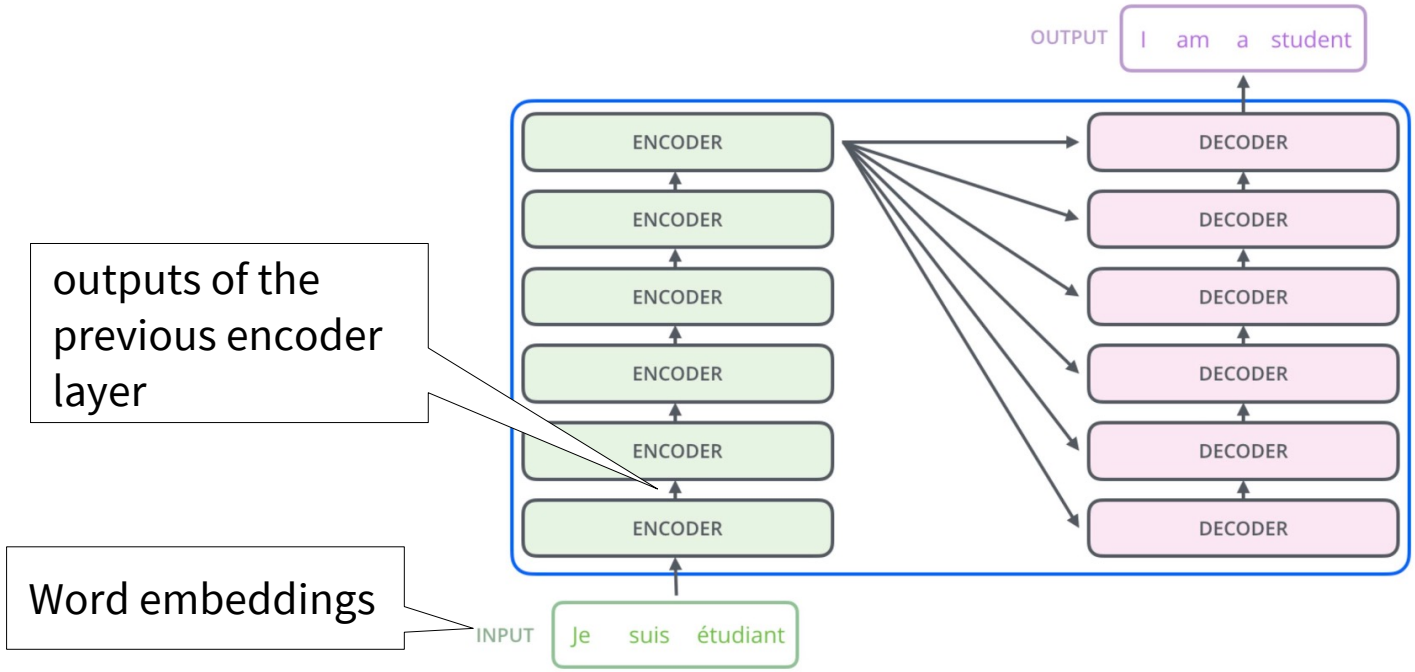
Transformer Architecture

- Opening the box with more details …



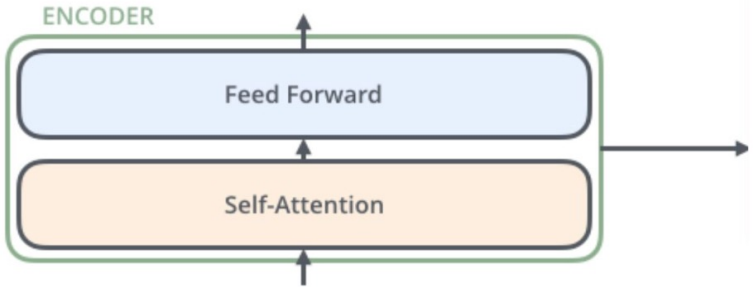
Transformer Architecture

- Opening the box with even more details …

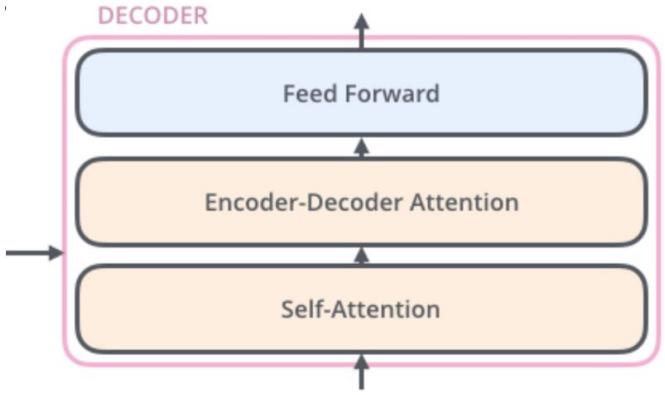


Transformer Architecture

Encoder:



Decoder:



Encoder:

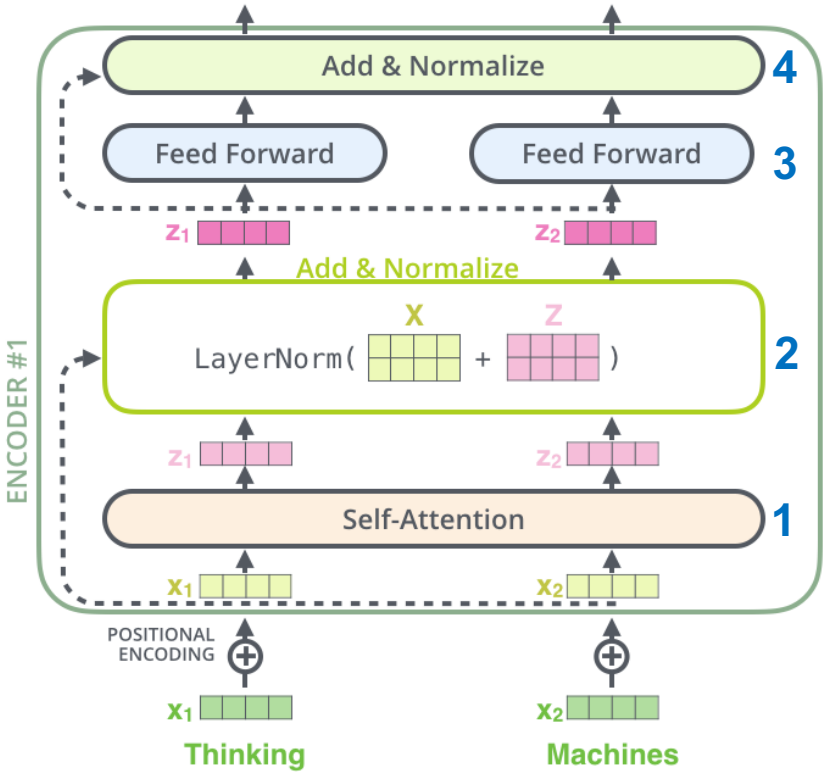
- Self-attention + Feedforward
- Produces contextual representations of input

Decoder:

- Masked self-attention
- Cross-attention (attend to encoder outputs)
- Generates output one token at a time

MANCHESTER 1824
The University of Manchester

Inner workings of the Encoder



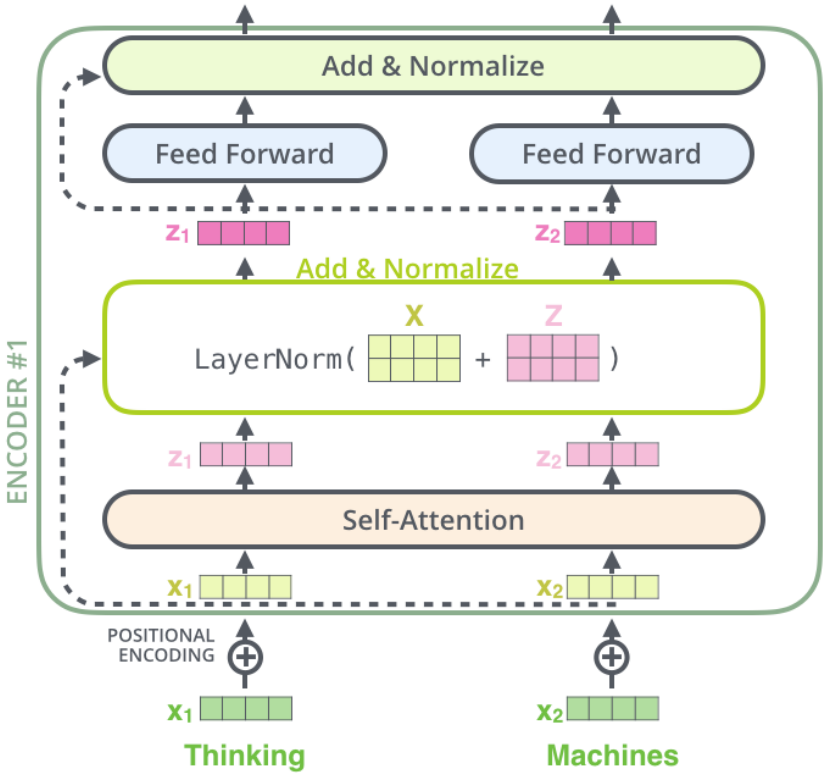
1. **Self-attention** mixes information across positions.
2. **Residual** + **LayerNorm**
3. **FNN** (position-wise) applies nonlinear transformation independently to each token.
4. **Residual** + **LayerNorm**

$$\text{LayerNorm}(x + \text{SelfAttention}(x))$$

$$\text{LayerNorm}(z + \text{FFN}(z))$$

MANCHESTER 1824
The University of Manchester

Inner workings of the Encoder



Important properties

- Tokens are processed in parallel within each layer
- Tokens are coupled via attention
- Deeper layers refine contextual representations
- Residual connections preserve signal and ease optimisation.

$$\text{LayerNorm}(x + \text{SelfAttention}(x))$$

$$\text{LayerNorm}(z + \text{FFN}(z))$$

MANCHESTER 1824
The University of Manchester

Self-Attention Mechanism

- Scaled Dot-Product Attention
- For input $X \in \mathbb{R}^{n \times d}$:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Interpretation:
  - *Q*, *K*, *V* are learned linear projections of the input
  - Attention scores $\frac{QK^T}{\sqrt{d_k}}$ measure similarity in projected space
  - Scaling $\sqrt{d_k}$ prevents dot products from growing large in high dimensions and stabilises gradients.
  - softmax converts attention scores into a distribution over the input elements
  - Output is a weighted sum of value vectors

MANCHESTER 1824
The University of Manchester

Understanding Self-Attention

- **Query (Q)**: The query vector represents the current input or the element for which we are trying to find relevant information.

  Example: Looking for action movies with strong female leads.

- **Key (K)**: The key vector represents the criteria or features against which the query is compared. Each element in the input data has an associated key vector.

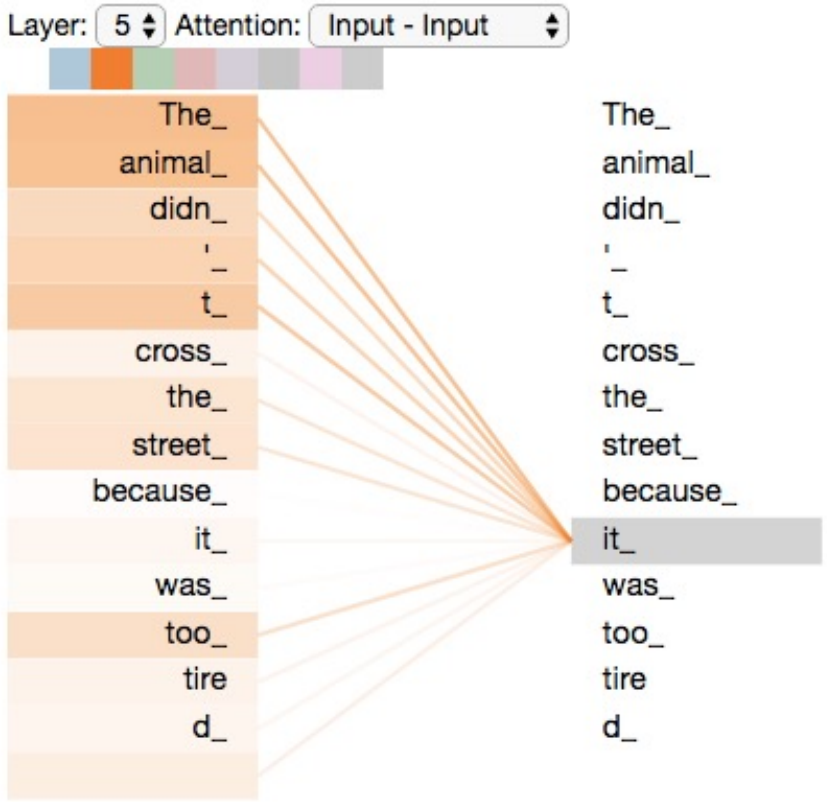
  Example: This movie is tagged as action, drama, female lead.

- **Value (V)**: The value vector holds the actual information or content that might be relevant to the query.

  Example: The full movie details or recommendation result.

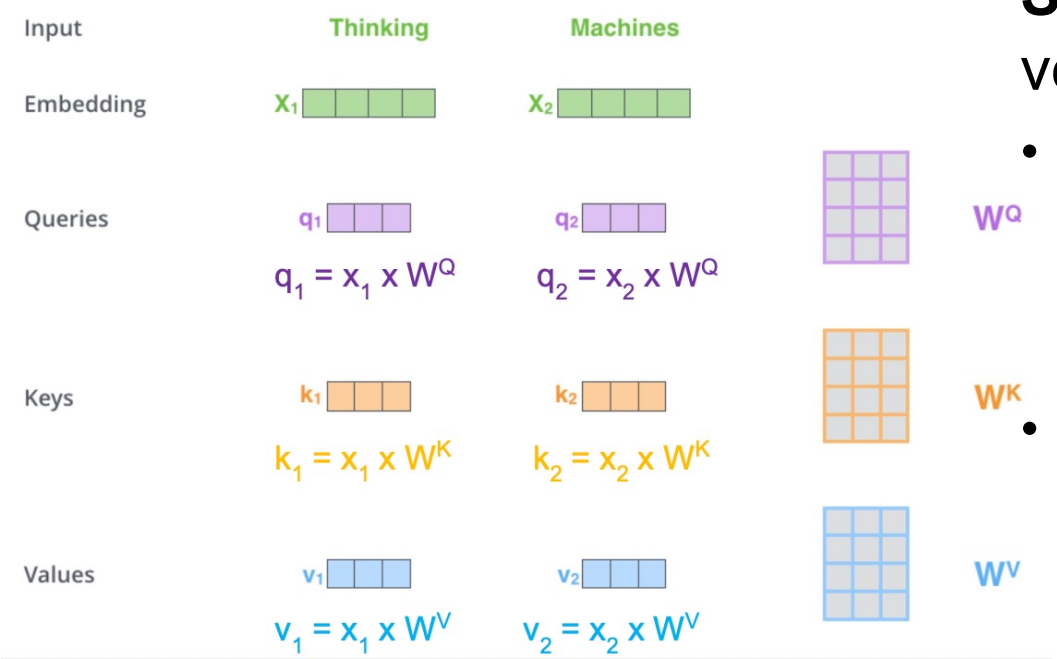
MANCHESTER 1824
The University of Manchester

Self-attention Visualisation



- Self-attention allows each token focus on relevant token in the sequence.
- Token "*it*" attends more to related tokens like "*The animal*"
- This helps the model capture long-range dependencies and context.

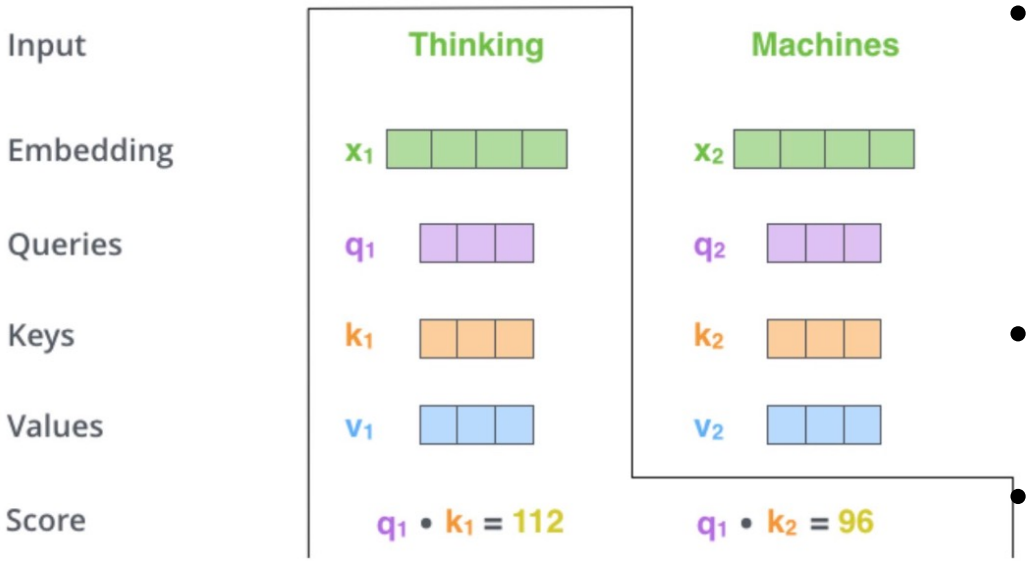
Calculating Self Attention



Step 1: Compute ***Q***, ***K***, and ***V*** vectors

- For each token, compute
  - a **query** vector
  - a **key** vector
  - a **value** vector.
- Derived by multiplying input embeddings with learned weight matrices

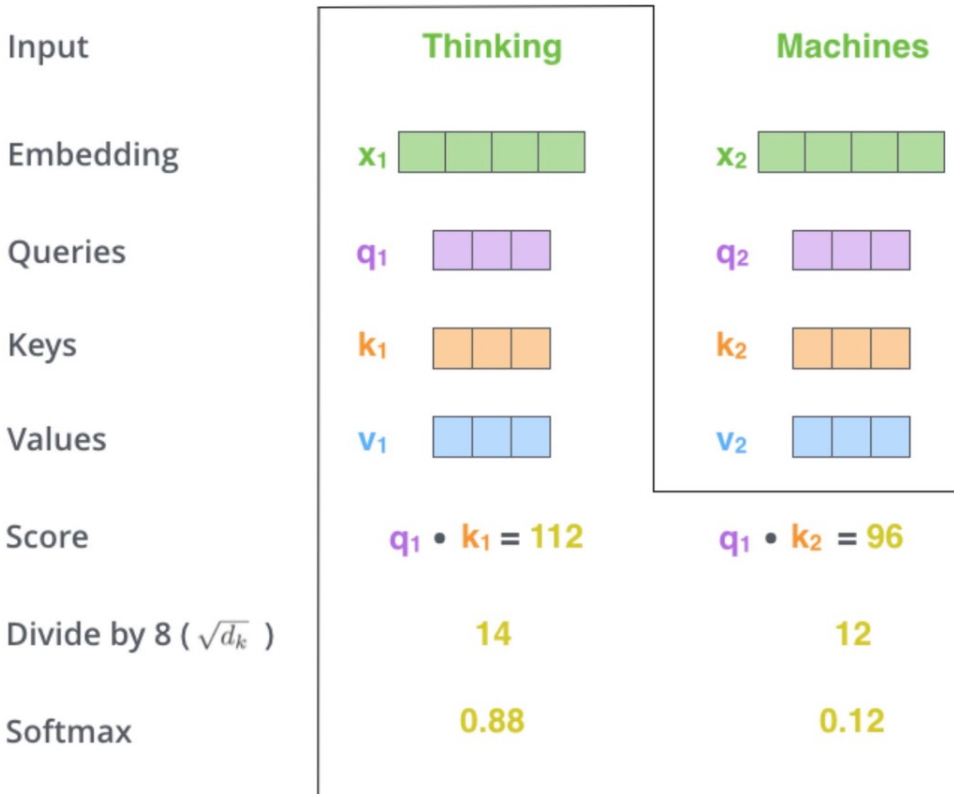
Calculating Self Attention



Step 2: Compute attention scores

- For each token, compute dot products between
  - Its query vector and all key vectors
  - e.g. $\boldsymbol{q_1}$ with $\boldsymbol{k_1}$, $\boldsymbol{k_2}$, $\boldsymbol{k_3}$, …
- This produces one score per token pair
- Larger dot product → stronger alignment

Calculating Self Attention

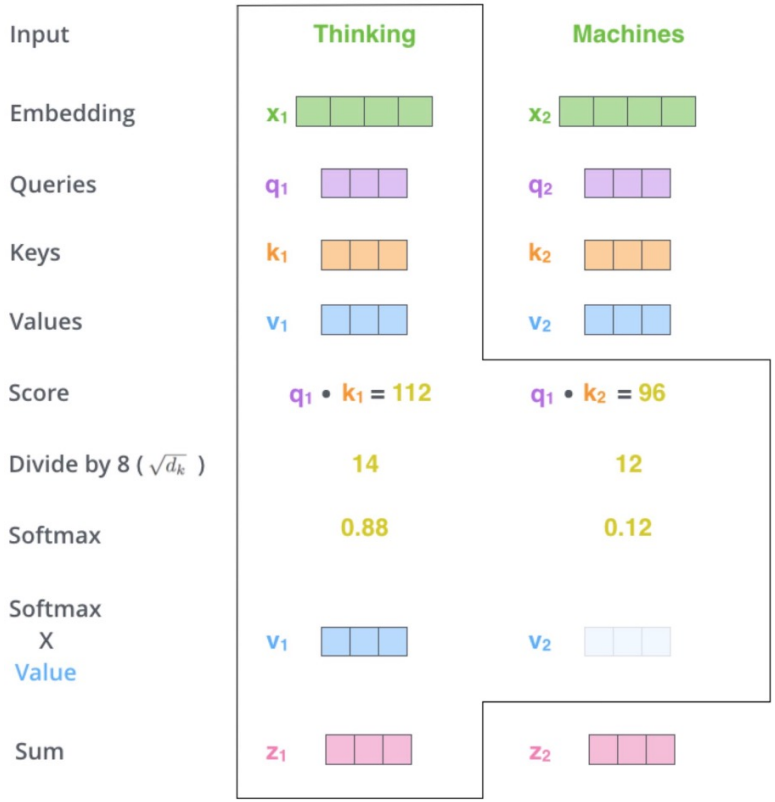


Step 3: Scale and normalize attention scores

- The dot product scores from Step 2 can become large, especially with high-dimensional vectors.
- Divide each score by the square root of the key dimension (e.g. $\sqrt{64} = 8$) to prevent large magnitudes
- This scaling helps stabilize gradients during training.
- Apply Softmax
  - All weights are positive
  - Converts to a probability distribution over tokens

MANCHESTER 1824
The University of Manchester

Calculating Self Attention

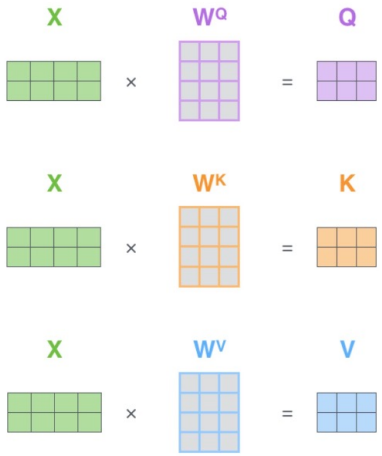


Step 4: Compute the self-attention output

- Multiply each value vector by its attention weight.
- This *emphasizes important* words and *down-weights irrelevant* ones.
- The result is the updated contextualized representation for the token
- This representation is then passed to the feed-forward layer for further processing

Efficient computation in matrix terms

- Calculate the *Query*, *Key*, and *Value* matrices for the entire context. We do that by packing the embeddings into a matrix X and multiplying it by the weight matrices.

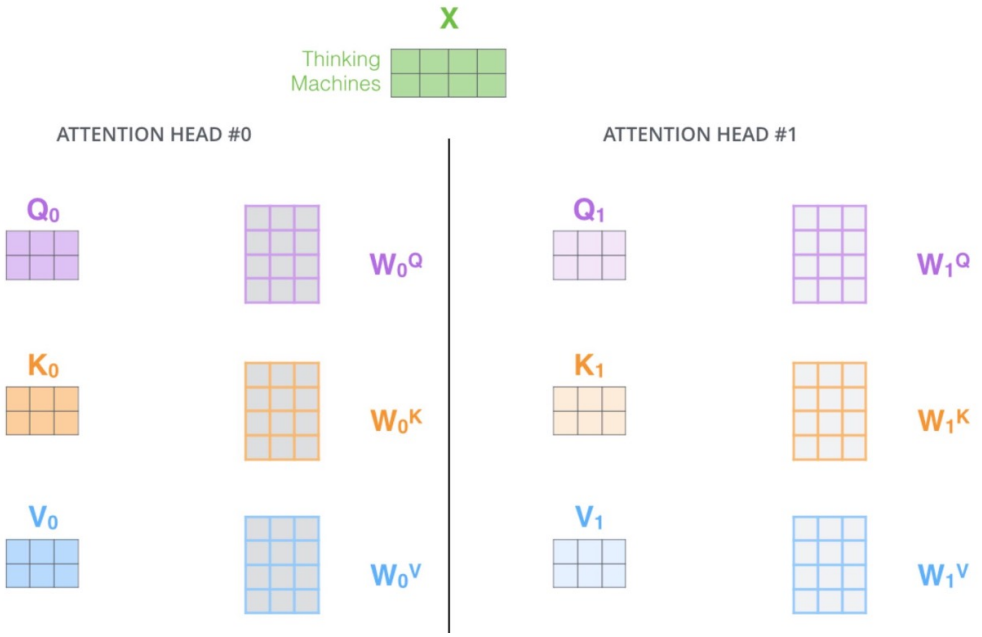


- Outputs of the self-attention layer:

$$Z = \text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$

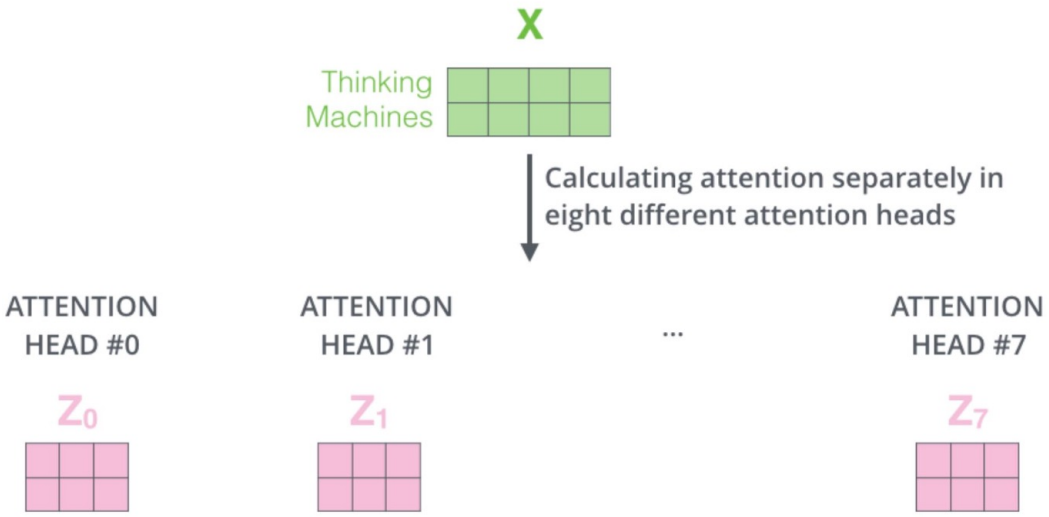
MANCHESTER 1824
The University of Manchester

A few more ornaments: Multiple heads!



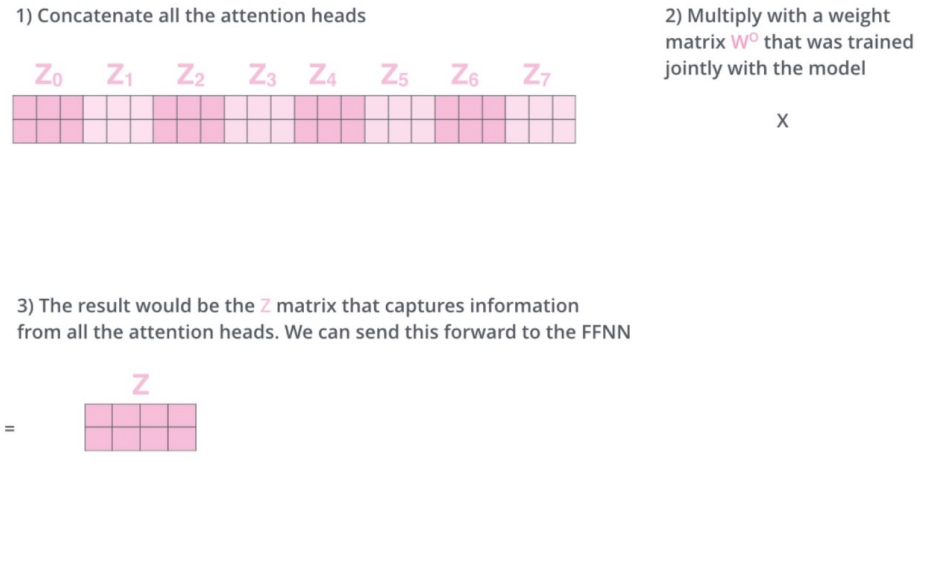
- Each self-attention layer computes multiple attention heads in parallel.
- Instead of one set of Q/K/V weights, we use multiple sets one per head, i.e. "***multi-head attention***"
- Each head learns to capture different aspects or patterns in the data.
- **Intuition**: enables the model to attend to different representation subspaces simultaneously (e.g. *river 'bank'* vs. *financial 'bank'*)

A few caveats: Multiple heads!



- Each attention head produces its own output matrix (Z_0, Z_1, ..., Z_7).
- These outputs are computed independently in different learned projection subspaces and can model diverse relational patterns from the same input.

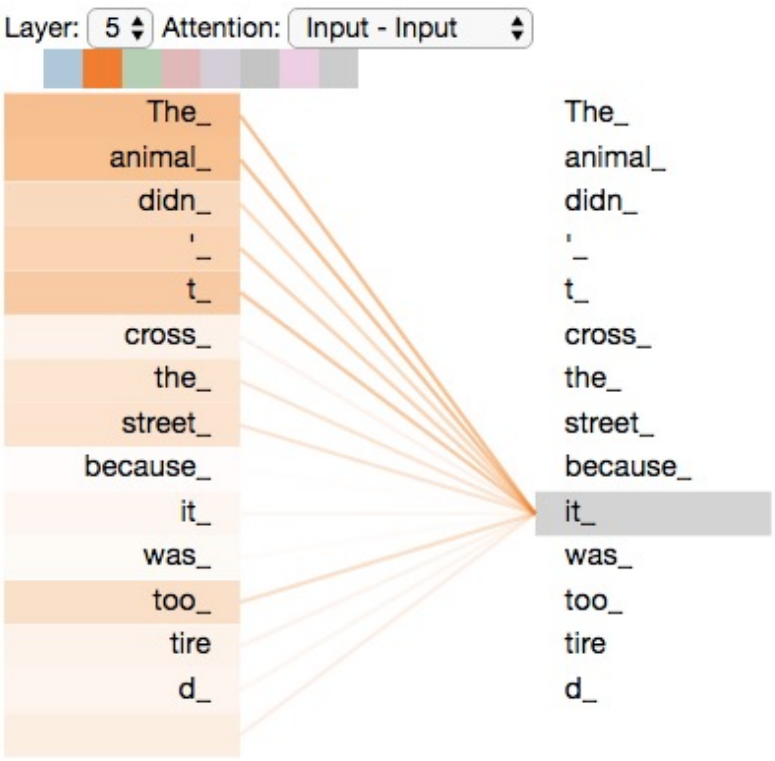
Combining Outputs from Multiple Heads

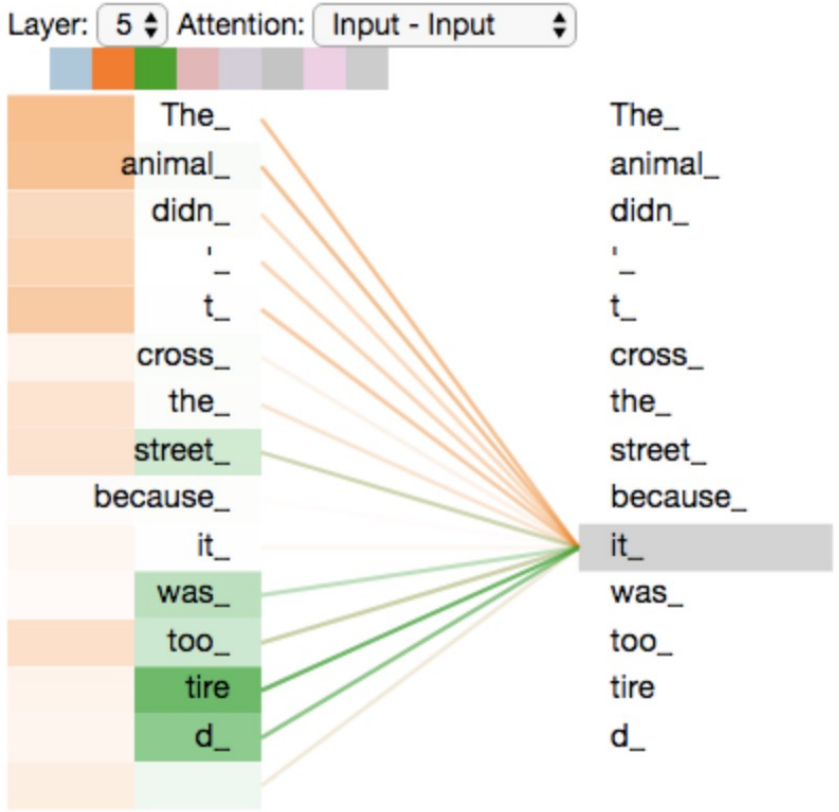


- Concatenate the outputs (Z_0, Z_1, ..., Z_7) from all attention heads.
- Multiply the combined matrix by a learnable output projection matrix *Wo*.
- This gives the final output *Z*, which integrates diverse attention patterns and is passed to the next layer.

MANCHESTER 1824
The University of Manchester

Multi-Headed Attention





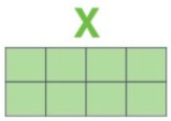
MANCHESTER 1824
The University of Manchester

All in one place…

1) This is our input sentence*

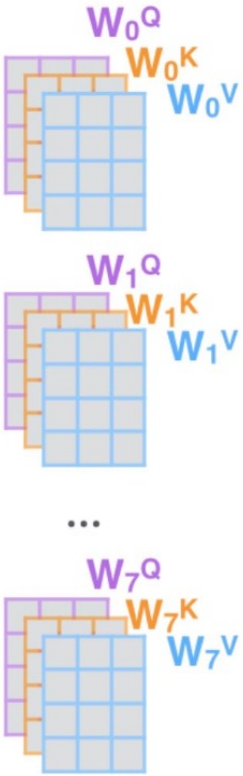
Thinking
Machines

2) We embed each word*

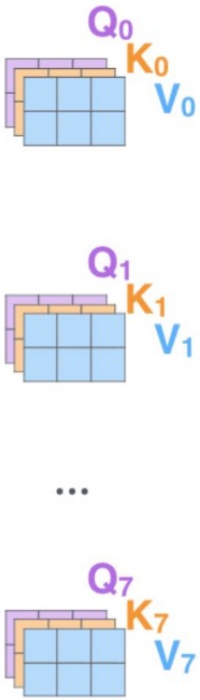


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

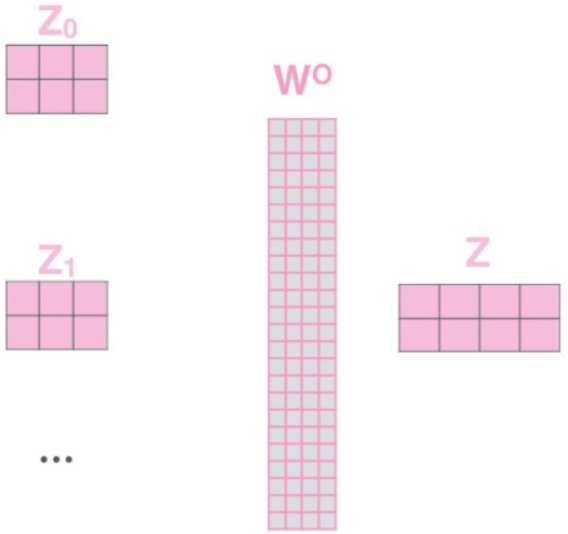
3) Split into 8 heads. We multiply X or R with weight matrices



4) Calculate attention using the resulting Q/K/V matrices



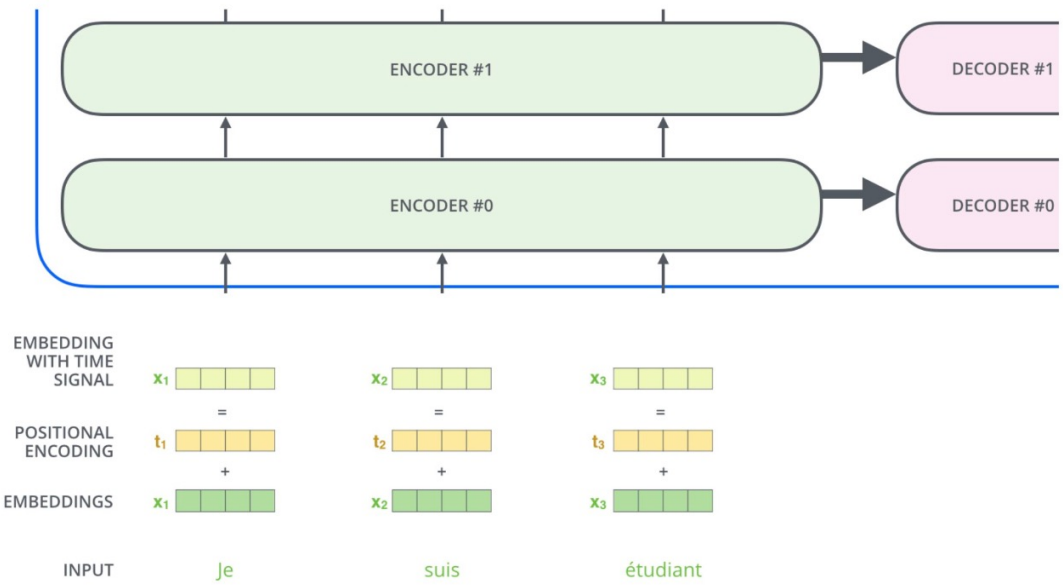
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer



MANCHESTER 1824
The University of Manchester

- What is missing so far …?

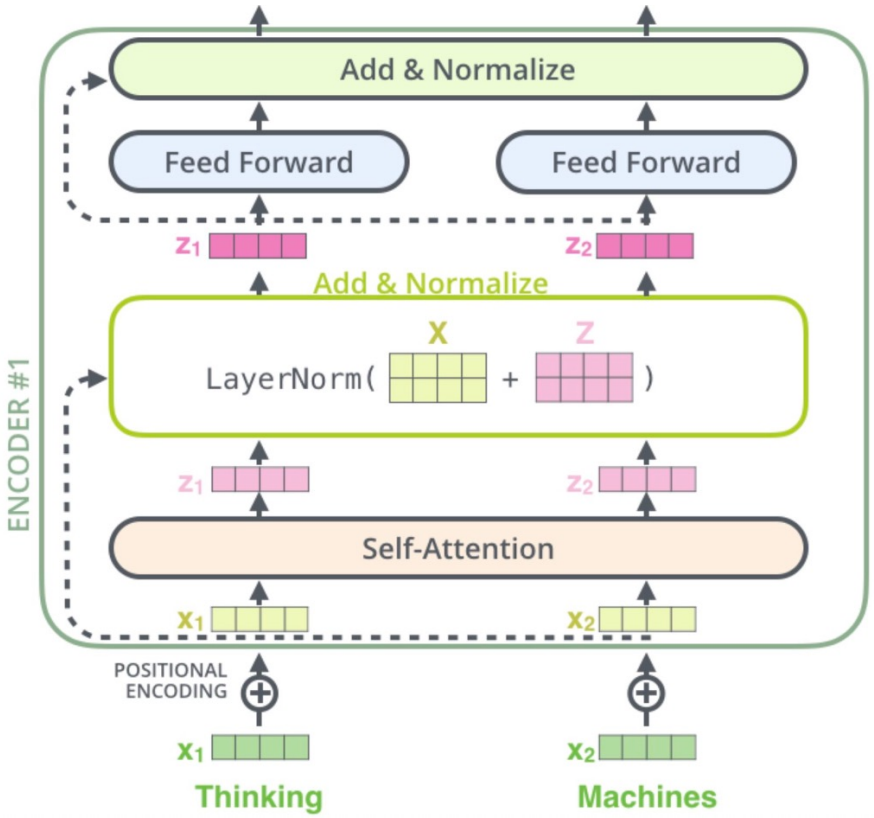
Another ornament: Positional encodings



- Transformers lack a built-in notion of word order.
- To encode position information, we **add a positional vector** to each input embedding.
- In the original Transformer, fixed sinusoidal positional encodings were used (i.e. sinusoidal functions of different frequencies)

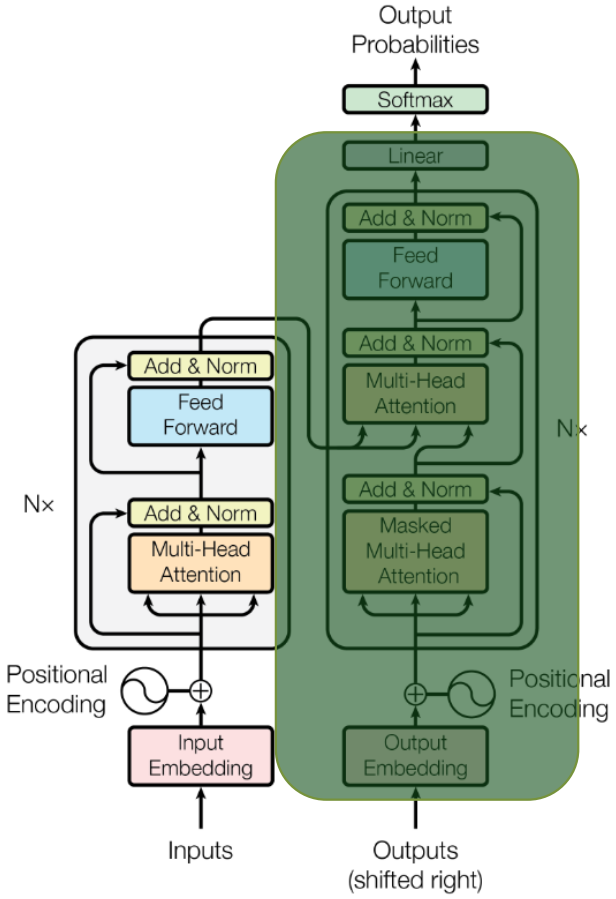
MANCHESTER 1824
The University of Manchester

Another ornament: Residuals



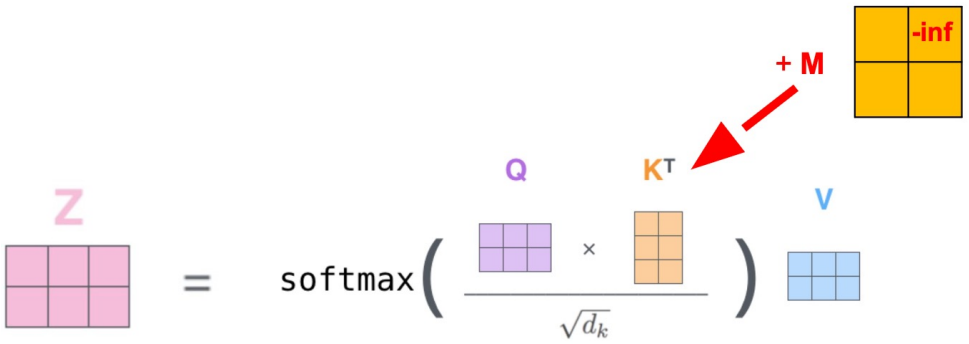
- To stabilize training in deep networks, each sub-layer is wrapped with a residual connection.
- In each sub-layer (self-attention or FFN), the input is added to the sub-layer output.
- The output is then passed through a layer normalization step.
- Residual connections improve gradient flow and facilitate optimization in deep architectures.

The Decoding Process



- **Cross Attention**: allows the decoder to attend to encoder outputs when generating each token
- **Output Generation**: each decoder layer combines the following to produce representations used to predict the next token
  - masked self-attention
  - encoder–decoder (cross) attention
  - a feed-forward network
- **Masked Self-Attention**: ensures each position can only attend to previously generated tokens (prevents access to future tokens)

Masking in the Decoder

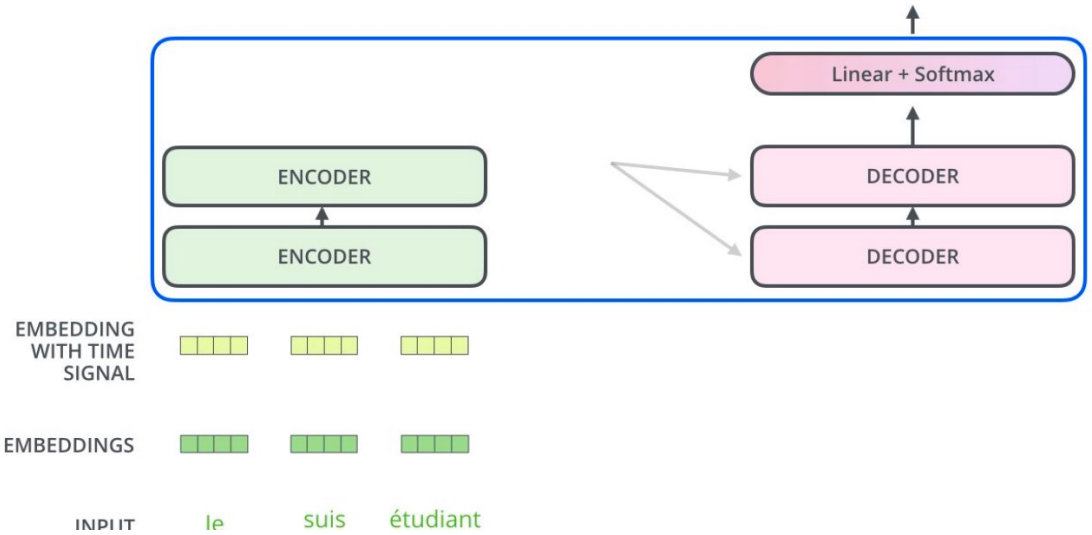


- The decoder's self-attention can only look at earlier tokens (i.e. not future ones).
- This prevents information leakage from future tokens during training.
- Implemented by masking future positions, setting their attention scores to -inf before softmax.

Generate token based on previous essentially

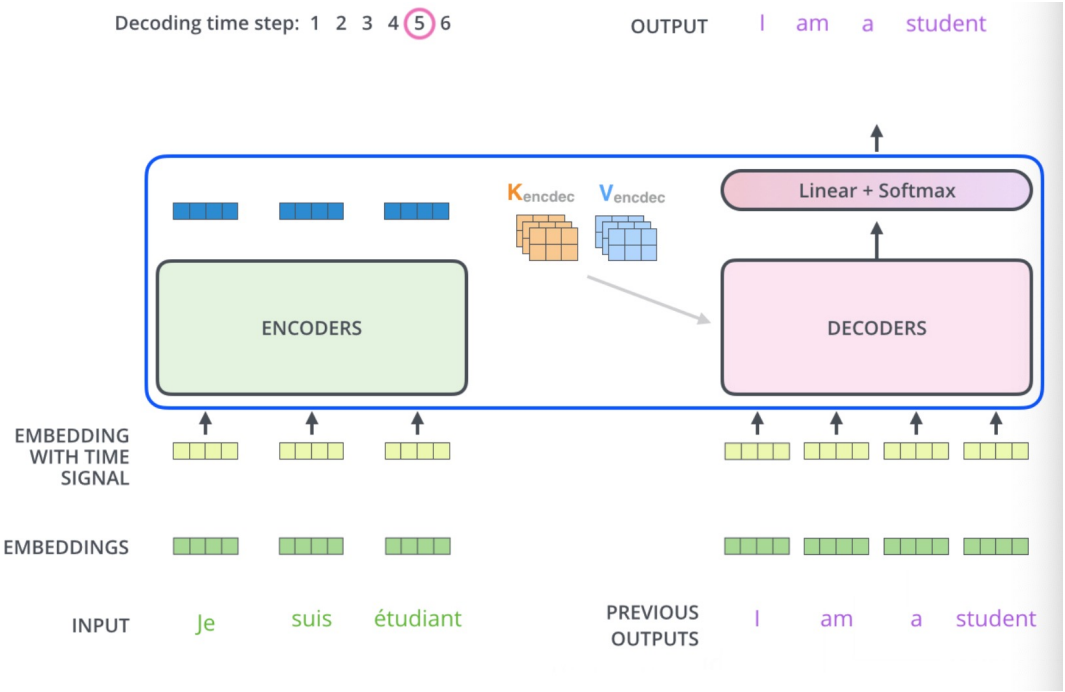
MANCHESTER 1824
The University of Manchester

Cross Attention



- The decoder attends to the encoder's output representations to generate predictions.
- The top encoder layer produces contextualized representations (used as **keys** and **values** in cross-attention).
- These are fed into the decoder's **cross-attention** layer.
- This helps the decoder **focus on relevant input positions** during generation.

From Decoder to Output Token



Use teacher-forcing to reinforce generating the next-token.

- **Training time:** the full target sequence is known. We use *teacher forcing*, feeding the correct previous tokens, to compute next-token predictions.
- **Inference time: o**utput is generated autoregressively, *one token at a time*. Each predicted token is fed back into the decoder.
- **Linear Layer**: Projects the decoder output to the vocabulary size.
- **Softmax Function**: Converts logits into probability distributions over the vocabulary.
- **Token Selection**: selects the token (e.g., via greedy decoding or sampling).

MANCHESTER 1824
The University of Manchester

How Transformers Learn: the Training Process

- **Loss Function**: cross-entropy loss is used to measure how far the predicted token probabilities are from the true target tokens. Lower loss indicates better predictions.
- **Backpropagation**: gradients of the loss are computed and used to update model parameters (e.g., via Adam optimizer).
- **Datasets**: trained on paired input–output sequences (e.g., source and target sentences for translation).

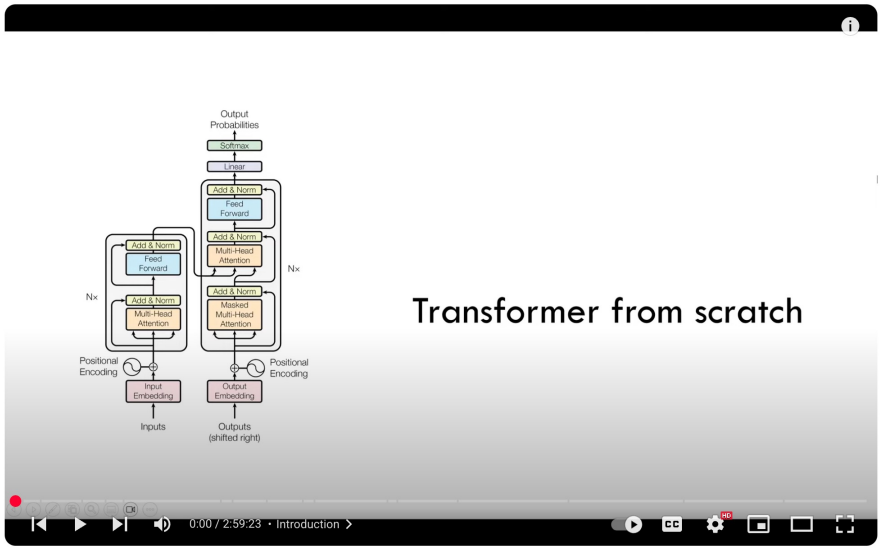
MANCHESTER 1824
The University of Manchester

Summary

- **Core Idea**: Overcomes limitations of RNNs and LSTMs (e.g., sequential bottlenecks, long-range dependencies) through global self-attention.
- **Key Components**: Self-Attention, Multi-Head Attention, Encoder-Decoder Structure, Positional Encoding
- **Why It Works**:
  – Short path length between tokens (i.e. $O(1)$ per layer)
  – Fully parallelizable across sequence positions
  – Scales effectively to very large models and datasets

MANCHESTER 1824
The University of Manchester

Building Transformers from Scratch



Coding a Transformer from scratch on PyTorch, with full explanation, training and inference.

Umar Jamil
69K subscribers
Subscribe
7.3K
Share